

Machines That Read

How language models read, write, and sometimes make things up

Day 08 · Week 2

Unlocks  Language Lab

1 Mission briefing

Your models can see. Now make one **read**. Language models do not look facts up in a database — they do one absurdly simple thing billions of times: guess the next token. Today you crack open GPT-2 and friends to watch tokens, embeddings, attention, and temperature at work — and catch a model sounding perfectly confident while being perfectly wrong. *Previously:* you unlocked  RPS Arena, transfer learning tested against your own hands.

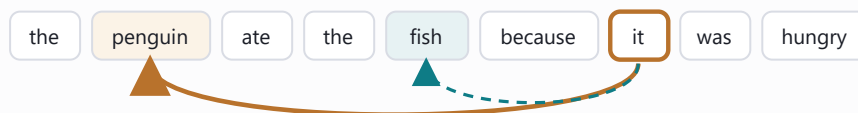
- ☐ Break text **tokens** (and see why an LLM into miscounts letters)
- ☐ Watch **next-token prediction**, the whole secret
- ☐ Peek at **attention**: which words look at which
- ☐ Turn words **embeddings** — meaning as into geometry
- ☐ Turn the **temperature** knob from robotic to unhinged
- ☐ Catch a **hallucination**: plausible ≠ true

2 Field guide the concepts

Tokens, not letters. A model reads **tokens** — common chunks of text, like puzzle pieces. "strawberry" might be two or three pieces, which is exactly why asking an LLM to count the letters in a word is weirdly hard for it.

Embeddings are meaning as geometry. Each token becomes a long arrow of numbers (768 of them for GPT-2). Similar meanings point similar directions — so much so that `king - man + woman ≈ queen`. And the entire engine is **next-token prediction**: given the text so far, score every possible next token, pick one, append it, repeat. Essays, code, and poems are all this one move on a loop, with **temperature** setting how boldly it picks.

Where does **it** point?



Reading "...the penguin ate the fish because **it** was hungry", attention links **it** back to **penguin** (and a little to **fish**).

Plausible ≠ true. A fluent, confident sentence can still be flat wrong — a **hallucination**. The model is trained to sound plausible, not to be correct. Always verify.

 **Matrix Watch** — attention is matrix multiplication too: every token's query vector times every token's key vector ($Q \cdot K^T$) is one big matmul — the very operation HPC engineers make fast.

3 Lab missions check each off in the notebook

1 Find a word that shatters ☐

Tokenize words until one splits into several pieces.

Expect: a familiar word broken into surprising sub-word tokens.

2 The strawberry problem 🍓 ☐

Ask a model to count a letter in a word, then check by hand.

Expect: it miscount — because it sees tokens, not letters.

3 Put your own words on the map ☐

Embed a few words and compare their vectors.

Expect: related words score as more similar than unrelated ones.

4 Predict from your own prompt ☐

Feed a story starter to a `text-generation` pipeline.

Expect: GPT-2 continue your sentence, token by token.

5 Fool the mood detector 😏 ☐

Craft a sentence that flips the sentiment classifier.

Expect: a sarcastic line the model reads the 'wrong' way.

6 Two temperatures 🌡️ ☐

Generate the same prompt at temperature 0.2 and 1.3.

Expect: safe-but-boring vs wild-but-creative text.

7 Follow the pronoun ☐

Use `fill-mask` to see what a pronoun refers to.

Expect: the link from 'it' back to the right noun.

4 Cheat sheet transformers pipelines

<code>tok = AutoTokenizer.from_pretrained("gpt2")</code>	Load GPT-2's tokenizer.
<code>tok.tokenize("strawberry")</code>	See the puzzle pieces a model reads.
<code>pipeline("text-classification", model=...)</code>	One-line sentiment / mood classifier.
<code>pipeline("text-generation", model="gpt2")</code>	Load GPT-2 as a text generator.
<code>gen(p, max_new_tokens=60, do_sample=True)</code>	Generate ~60 tokens with sampling on.
<code>temperature=0.9, pad_token_id=50256</code>	Creativity knob · GPT-2's end-of-text id.
<code>pipeline("fill-mask", model=...)</code>	Predict a masked-out word.
<code>s.replace("___", tok.mask_token)</code>	Swap your blank for the token the model expects.
<code>pipeline("zero-shot-classification", ...)</code>	Sort text into labels you invent at runtime.

5 Unlock your Studio module

Language Lab

Sign your work and set a default temperature; the transformer models load from the Hugging Face cache, so the unlock cell only writes:

```
ai_studio/models/language_lab.json
```

Four language superpowers — mood, story, fill-the-blank, sorting hat — go live:

```
$ python ai_studio/app.py
```

6 Mini-glossary

token

A chunk of text (word or word-piece) the model actually reads.

transformer

The attention-based architecture behind modern language models.

temperature

The knob controlling randomness in generation.

hallucination

A fluent, confident output that simply is not true.

embedding

A token turned into a vector; meaning becomes direction.

attention

The mechanism letting each token weigh the others for context.

LLM

Large language model — a giant next-token predictor.

7 Show & tell

1. Show a word your tokenizer split weirdly. Why does that make letter-counting hard?
2. Same prompt at low vs high temperature — which felt more useful, and for what?
3. Show a confident hallucination you caught. How would you fact-check a model in real life?